



Contents lists available at ScienceDirect

Journal of King Saud University – Computer and Information Sciences

journal homepage: www.sciencedirect.com

Cellular automata-based approach for salt-and-pepper noise filtration

Zubair Jeelani, Fasel Qadir*

Department of Computer Sciences, University of Kashmir, India

ARTICLE INFO

Article history:

Received 23 July 2018

Revised 26 November 2018

Accepted 20 December 2018

Available online 22 December 2018

Keywords:

Salt-and-pepper noise

Cellular automata

Structural similarity index

Peak signal to noise ratio

Noise filtration

ABSTRACT

Application of cellular automata (CA) to digital image processing has achieved considerable attention in last several years. CA are now employed in noise filtration of digital images; particularly many CA-based impulse noise filters have been proposed. Salt-and-pepper noise is a special kind of impulse noise and is introduced in an image during transmission over transmission media by external noise sources like atmospheric disturbances, or due to corrupted hardware memory locations or fault in camera sensors. In this paper, we present five salt-and-pepper noise filters based on modifications of outer totalistic cellular automata (OTCA) with the adaptive neighborhood. Use of OTCA model makes the proposed filters computationally simple on one hand and the use of adaptive neighborhood help the filters to provide efficient noise filtration at varying noise densities on the other. Comparative analysis of these filters followed by comparison with several standard and CA-based filters in terms of peak signal to noise ratio (PSNR) and structural similarity (SSIM) index is presented.

© 2018 The Author(s). Published by Elsevier B.V. on behalf of King Saud University. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

1. Introduction

Digital images find tremendous use in a wide variety of areas and the increase of digital image use may be attributed to rapid growth of information technology, which has facilitated gradual replacement of traditional form of text data with image data in many applications (Thepade et al., 2017). Digital images sometimes carry sensitive information as in case of health-care medical reports that take the form of digital images. Noise in image processing generally refers to unwanted data that conceal or distort the information carried in digital images. Images may get corrupted by different types of noise during different stages of image processing like image acquisition due to malfunctioning of the sensors in a digital camera, or during encoding and transmission, when the images are transferred over noisy transmission lines. It becomes immensely vital to remove this noise because presence of noise may lead to misleading information carried in digital images and adversely affect the subsequent decisions to be taken

on the basis of this information. Computer vision systems are also sensitive to presence of noise in digital images and fail to produce appropriate results if the digital image is not filtered for noise (Qadir and Shoosha, 2018). As such, image restoration has become one of the most investigated fields of image processing where the original image is recovered from the noisy image (Singh et al., 2018). Different types of noise like Impulse, Gaussian, or Speckle noise may corrupt digital images. There are two types of impulse noise, salt-and-pepper noise and random valued noise. For images corrupted by salt-and-pepper noise, the noisy pixels can take only the maximum and minimum values in the dynamic range (Sahin et al., 2014). Random valued noise on the other hand, may assume any value in the dynamic range. Although, many linear and non-linear filters have been proposed in literature but they tend to perform poor as the noise densities increase in the digital images. Lack of adaptability in these filters to varying densities of noise has paved way for the application of CA to noise filtration. In last few years, many CA-based noise filtration algorithms have been proposed. Many of these filters use variations of totalistic cellular automata (TCA) while others are based on fuzzy cellular automata (FCA).

Linear filters, like average filter (AF), for removing of impulse noise are not satisfactory (Sahin et al., 2014) because their denoising performance deteriorates with increase in noise density and these filters, in addition to noise, remove useful information from the images as well. One of the most popular non-linear filters is the median filter (MF) which results in effective filtering when the noise density in an image is low. MF provide better detail

* Corresponding author at: Department of Computer Sciences, North Campus, University of Kashmir, Delina, Baramulla, Jammu and Kashmir 193103, India.

E-mail address: fasel.scholars@gmail.com (F. Qadir).

Peer review under responsibility of King Saud University.



Production and hosting by Elsevier

preservation in the image when compared to AF. The standard mean filter (Tukey, 1974) works on every pixel in the input image irrespective of if it is corrupted with noise or not. Processing uncorrupted pixels modify their original value which may lead to information loss. This proves very expensive sometimes if, for instance, the image contains some watermark. As such, many variations of standard median filter are proposed. The weighted median filter (WMF) (Brownrigg, 1984) and center weighted median filters (CWMF) (Ko and Lee, 1991) improve the de-noising performance of the standard mean filter by assigning more weight to the particular pixel within the kernel. All filters discussed above process all pixels and, as such, the loss of original information in the uncorrupted pixels cannot be avoided. Switching median filter (SMF) (Sun and Neuvo, 1994) use median filtering only on corrupted pixels, therefore, improving the noise filtering performance. SMF, however, fail to show consistency in its noise filtering capabilities as the noise density increases in the given image. Non adaptability of these filters to varying densities of noise makes them perform badly as the noise density increases.

Liu et al. (2008) proposed a salt-and-pepper noise filter based on CA with Moore neighborhood (N_M). The noise detection is done by obtaining minimum and maximum intensity value in N_M of the current pixel. If the intensity value of current pixel is between this range, the pixel is left unprocessed, otherwise the pixel is processed. Use of 8-neighbors (Moore neighborhood) of a pixel help achieve good results for low noise density but as the noise density increases, this method produce distorted filtered image. Selvapeter and Hordijk (2009) proposed a CA based image noise filtration algorithm. The “majority” CA update rule is used that is used to assign the intensity value of majority of cells in the local neighborhood of the corrupted cell. Use of fixed 4-neighbors (von Neumann neighborhood) of a pixel leaves the noise filtration model with very less choices as the noise becomes high in the image. As such, with high occurrence of noise, the method fails to provide good results. Dalhoum et al. (2011) proposed a CA-Based Filter for two types of noise; salt-and-pepper noise and uniform noise. Sadeghi et al. (2012) proposed an efficient impulse noise filter based on fuzzy cellular automata. The model obtains noise density by using average and standard deviation of central and neighboring cells. First, the CA-based noise detection is used to detect impulse noise in the image and if the image is noisy then FCA is used for noise filtration. Sahin et al. (2014) proposed FCA-based salt-and-pepper noise filtering method. Noise detection is done by considering the maximum and minimum pixel values in the selected window and the value of current pixel is compared with these values. If the current pixel value is out of this range, the pixel is processed. Shukla and Agarwal (2014) have proposed a CA-Based noise filtration algorithm. Local transition function uses N_M with $r = 1$ and the experiment is applied to gray-scale images. A factor α which belongs to $[0, 0.5]$ is used in the local transition function. Sargolzaei et al. (2014) proposed FCA-based impulse noise reduction method. A fuzzy filtering operator is used to identify the noise in a pixel using its neighboring pixels. The correction value of the corrupted pixel is then calculated using the dynamic CA modeling approach. Pathak et al. (2016) proposed a CA-based technique to suppress the speckle noise in ultrasound images using non-local mean. CA with Moore neighborhood and mean values of different neighboring regions are computed. The Maximum Likelihood Neighborhood Condition (MLNC) rules are then applied to find the nearest value in the neighbors. Tourounis et al. (2018) proposed a salt-and-pepper noise filtering method using CA employing N_M with $r = 1$ for CA evolution. This method considers 8-bit grayscale images and use two thresholds, 0 and 255. If the current pixel/ cell is corrupted, the next state is computed using the average of those cells in M_N with $r = 1$ that does not have either state 0 or 255. Qadir and

Shoosha (2018) proposed an efficient CA-based impulse noise filtration method that uses primary CA rules using the Moore neighborhood for noise filtration but in case of high density impulse noise all the cells in Moore neighborhood may be corrupted by noise in which case the method uses extended Moore (25-cell) neighborhood and the “majority update” rule for the de-noising process. Roy et al. (2016) proposed an impulse noise filter employing SVM (support vector machine) fuzzy filter for grayscale images. Kumar and Nagaraju (2018) proposed a technique for impulse noise identification and removal from grayscale images. SVNN (support vector neural network) based fuzzy hybrid filter is used to identify and remove the noise from the grayscale images.

This paper apply simple two dimensional OTCA model to salt-and-pepper noise filtration and make an adaptive neighborhood model that adapts and responds to density of noise around the corrupted pixel. Proposed model processes pixels (cells) asynchronously so that only the values of corrupted pixels are changed. Using OTCA asynchronously makes the model computationally very efficient and using an adaptive neighborhood help to provide better filtration even at high noise density.

Rest of this paper is organized as follows: Section 2 describes the concept of CA and OTCA; in section; Section 3 presents the proposed noise filtration model; Section 4 provides the detailed results and discussion followed by conclusion and future scope of the study in Section 5.

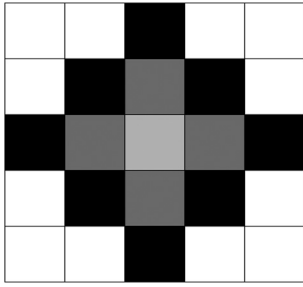
2. Cellular automata

Cellular Automata (CA) are time and space discrete mathematical models described in late 1940s by John von Neumann and Stanislaw Ulam. CA have been considered one of the most pronounced parallel computational tools in the recent era of nature and bio-inspired computing (Tourounis et al., 2018). Because of their local interaction, simplicity of design and inherent parallelism, many hardware implementations of CA have been realized in literature (Preston and Duff, 1984; Halbach and Hoffmann, 2004; Toffoli and Margolus, 1987). Therefore, CA are ideal models that can be applied to many image processing tasks requiring pixel processing in the spatial domain.

The space in CA is divided into discrete fundamental space units called cells and each cell possess a state at any given time t . CA evolve to produce the new generation of CA at time $t + 1$ by changing the state of its cells according to a pre-defined rule (transition function). This rule determines the new state of each cell as a function of the current state of the cell and the states of the cells in its neighborhood (Jeelani and Qadir, 2018). Cells within a CA may occur in one, two, three or even in higher dimensions. However, for our purposes we consider only 2D-CA henceforth. There are many neighborhood conditions possible for a 2D-CA but the two most used neighborhood conditions in literature are the von Neumann (N_{vN}) which includes 4 non-angular adjacent cells to the central cell and the Moore neighborhood (N_M) which includes all 8 adjacent neighboring cells (4 on sides and 4 on corners) around the central cell. A special class of CA called the outer totalistic cellular automata (OTCA) are those CA in which the new state of a cell $C(i, j)$ is computed as a function (δ) of state of the cell under observation and states of the cells in its neighborhood taken separately (Packard and Wolfram, 1985).

3. Proposed method

Pixels in images corrupted with salt-and-pepper noise take either the minimum or maximum intensity value in the dynamic range. For 8-bit grayscale images, therefore, the possible values

Fig. 1. Cells in N_M with $r = 2$.

Ψ_{21}	Ψ_{13}	Ψ_9	Ψ_{14}	Ψ_{22}
Ψ_{15}	Ψ_5	Ψ_1	Ψ_6	Ψ_{16}
Ψ_{10}	Ψ_2	$C(i,j)$	Ψ_3	Ψ_{11}
Ψ_{17}	Ψ_7	Ψ_4	Ψ_8	Ψ_{18}
Ψ_{23}	Ψ_{19}	Ψ_{12}	Ψ_{20}	Ψ_{24}

Fig. 2. Primary rule mapping to neighborhood cells of $C(i,j)$.

are 0 (pepper noise) and 255 (salt noise). The proposed model detects the noisy pixel by checking it for these extreme values. The model use intensity values of adjacent pixels around the corrupted pixel for filtration when the noise density is less. As the noise density increase, the adaptive neighborhood lets the model to use intensity values of non-local pixels for filtration. In Fig. 1, the central cell (light gray) and four cells (dark gray) around it constitute the von Neumann neighborhood (N_{vN}) with radius $r = 1$. N_{vN} with $r = 1$ and the cells in black color constitute N_{vN} with $r = 2$. The cells in N_{vN} of a cell at location (i,j) with $r \geq 1$ are represented by Eq. (1) (Dalhoum et al., 2012).

$$N_{vN}(i,j,r) = \{(i',j') : |i' - i| + |j' - j| \leq r\} \quad (1)$$

Moore neighborhood (N_M) represents a square lattice of cells and is composed of a central cell surrounded by neighborhood cells with radius, r . In Fig. 1, 12 white cells in addition to the cells in N_{vN} with $r = 2$ constitute N_M with $r = 2$. We specify the cells in N_M of a cell at location (i,j) with $r \geq 1$ (Dalhoum et al., 2012) as shown in Eq. (2).

$$N_M(i,j,r) = \{(i',j') : |i' - i| \leq |j' - j| \leq r\} \quad (2)$$

The input image C , possibly corrupted with salt-and-pepper noise is the initial CA configuration and the local transition function (δ) computes the next state of cell $C(i,j)$, at time $t + 1$, as a function of the current state of $C(i,j)$ and state of a single cell in its adaptive neighborhood at time t . A cell from the neighborhood is chosen according to a primary rule (ψ_n). The general form of the local transition function (δ) takes the form as shown in Eq. (3).

$$C(i,j)^{t+1} = \delta(C(i,j)^t, C(i',j')^t) \quad (3)$$

where,

$$C(i',j') \in N_M(i,j,r)$$

$$C(i,j) = 0 \vee C(i,j) = 255 \text{ and } C(i',j') \neq 0 \wedge C(i',j') \neq 255$$

$$C(i,j)^t \text{ is the noisy pixel, and } C(i,j)^{t+1} \text{ is the filtered pixel.}$$

As shown in Eq. (3), δ is applied to a cell only if it has value 0 or 255 and, as such, C is in essence an asynchronous cellular automaton (Manzoni, 2012).

The primary rules (ψ_n) are defined for a cell $C(i,j)$ such that the next state of this cell depends on the state of only one cell in its neighborhood. Mapping of primary rules with neighborhood cells in $N_M(i,j,2)$ of a cell $C(i,j)$ are shown by placing primary rules in corresponding cells in Fig. 2. The primary rules select a neighborhood cell that is in turn used by δ to calculate next state of cell $C(i,j)$. The definition of δ can be, therefore, rewritten as Eq. (4):

$$C(i,j)^{t+1} = \delta(C(i,j)^t, \psi_n(C(i,j)^t)) \quad (4)$$

Filters discussed in the following subsections use CA-based salt-and-pepper noise filter presented in Eq. (3). These filters, however, differ on the basis of neighboring pixels/cells they consider and/or the order in which these pixels/cells are considered.

3.1. Filter 1 (F1)

This filter uses primary rules ψ_1 to ψ_{12} to assign new value to $C(i,j)$ according to the local transition function (δ) given in Eq. (3) with $C(i',j') \in N_{vN}(i,j,2)$. Procedure to select $C(i',j')$ is given under Algorithm 1.

Algorithm 1 (C)

```

1: for all  $(i,j)$  in  $C$  do
2:   if  $C(i,j) = 0$  or  $C(i,j) = 255$  then
3:     for  $k \leftarrow 1$  to 12 do
4:       if  $\psi_k(C(i,j)) \neq 0$  and  $\psi_k(C(i,j)) \neq 255$  then
5:          $C(i',j') = \psi_k(C(i,j))$ 
6:       end for
7:     end if
8:   end for
9: end if
10: end for

```

In Algorithm 1, the pixel/cell $C(i,j)$ is tested for salt-and-pepper noise and if the pixel is corrupted, intensity value of the pixel corresponding to ψ_k is assigned to $C(i,j)$ using δ . But if $\psi_k(C(i,j))$ fails i.e. corresponding cell state of ψ_k is either 0 or 255, primary rule $\psi_{k+1}(C(i,j))$ is used.

3.2. Filter 2 (F2)

Filter 2 use same primary rules (ψ_n) under local transition function (δ) as used in F1 and presented in Algorithm 1; but in this filter, pixels are processed from bottom to top and from right to left whereas in F1, pixels are processed from top to bottom and from left to right.

3.3. Filter 3 (F3)

Only two primary rules ψ_1 and ψ_4 are used to assign new value to $C(i,j)$ according to δ given in Eq. (3) with $C(i',j') \in \{C(i-1,j), C(i+1,j)\}$. Procedure to select $C(i',j')$ is given under Algorithm 2.

Algorithm 2 (C)

```

1: for all  $(i,j)$  in  $C$  do
2:   if  $C(i,j) = 0$  or  $C(i,j) = 255$  then
3:     if  $\psi_1(C(i,j)) \neq 0$  and  $\psi_1(C(i,j)) \neq 255$  then
4:        $C(i',j') = \psi_1(C(i,j))$ 
5:     else
6:       if  $\psi_4(C(i,j)) \neq 0$  and  $\psi_4(C(i,j)) \neq 255$  then
7:          $C(i',j') = \psi_4(C(i,j))$ 
8:       end if
9:     end if
10:  end if
11: end for

```

The pixel/cell $C(i,j)$ is tested for salt-and-pepper noise in bottom up order and if the pixel is corrupted, the intensity value of pixel corresponding to ψ_1 is assigned to $C(i,j)$ using δ . But if $\psi_1(C(i,j))$ fails, primary rule $\psi_4(C(i,j))$ is applied to $C(i,j)$ as shown in Algorithm 2. If both $\psi_1(C(i,j))$ and $\psi_4(C(i,j))$ fail, the cell $C(i,j)$ is left unchanged.

3.4. Filter 4 (F4)

This filter use primary rules ψ_1 to ψ_{24} to choose a new value for $C(i,j)$ using the local transition function (δ) with $C(i',j') \in N_M(i,j,2)$. Procedure to select $C(i',j')$ is given under Algorithm 3.

Algorithm 3 (C)

```

1:  $l \leftarrow 1$ 
2: for all  $(i,j)$  in  $C$  do
3:   if  $C(i,j) = 0$  or  $C(i,j) = 255$  then
4:     for  $k \leftarrow 1$  to 24 do
5:       if  $\psi_l(C(i,j)) \neq 0$  and  $\psi_l(C(i,j)) \neq 255$  then
6:          $C(i',j') = \psi_l(C(i,j))$ 
7:          $l = \text{nextrule}(l)$ 
8:       end for
9:     end if
10:     $l = \text{nextrule}(l)$ 
11:  end for
12: end if
13: end for

```

Algorithm 4 nexrule(l)

```

1: if  $l = 2$  then
2:    $l \leftarrow 6$ 
3: else  $l = l + 1$ 
4: end if
5: if  $l \geq 24$  then
6:    $l = 1$ 
7: end if
8: return  $l$ 

```

The pixel/cell $C(i,j)$ is tested for salt-and-pepper noise in top down order and whenever a pixel $C(i,j)$ is found to be corrupted by salt-and-pepper noise, it is replaced by pixel values mapped by rule ψ_l . If ψ_l fails, then ψ_{l+1} is used to select $C(i',j')$. But if the last pixel was treated with primary rule ψ_{24} , then next pixel is treated starting with ψ_6 out of turn.

3.5. Filter 5 (F5)

Primary rules ψ_1 to ψ_{12} are used to assign new value to $C(i,j)$ in a rotational order according to the local transition function (δ) given in Eq. (3) with $C(i',j') \in N_{vN}(i,j,2)$. Procedure to select $C(i',j')$ is given under Algorithm 5.

Algorithm 5 (C)

```

1:  $l \leftarrow 1$ 
2: for all  $(i,j)$  in  $C$  do
3:   if  $C(i,j) = 0$  or  $C(i,j) = 255$  then
4:     for  $k \leftarrow 1$  to 12 do
5:       if  $\psi_l(C(i,j)) \neq 0$  and  $\psi_l(C(i,j)) \neq 255$  then
6:          $C(i',j') = \psi_l(C(i,j))$ 
7:          $l \leftarrow l + 1$ 
8:       if  $l \geq 12$  then
9:          $l \leftarrow 1$ 
10:      end if
11:    end for
12:  end if
13:   $l \leftarrow l + 1$ 
14:  if  $l \geq 12$  then
15:     $l \leftarrow 1$ 
16:  end if
17: end for
18: end if
19: end for

```

All pixels/cells are tested for salt-and-pepper noise in top down order and if the pixel is corrupted, the intensity value of pixel corresponding to $\psi_k(C(i,j))$ is assigned to $C(i,j)$. Next corrupted pixel in row-first order is processed according to the procedure given under Algorithm 5 starting with $\psi_{k+1}(C(i,j))$.

4. Results and discussion

Matlab R2016a is used to implement all the filters and dataset of 35 images of varying size is used in all the experiments. These images are first converted to 8-bit grayscale format wherever necessary and then each grayscale image is corrupted with 5% to 95% salt-and-pepper noise ratios. As such, 10 salt-and-pepper noise corrupted versions of each image are obtained. In Table 1, first 25 images in the dataset are chosen at random from the Berkeley Segmentation Dataset and Benchmark (www.eecs.berkeley.edu/Research/Projects/CS/vision/bsds), next six photographs in the table are captured by authors of this work using a smart phone and the remaining four are standard test images. Fig. 3 show the subjective results of filtering *lena* test image using proposed filters with noise density varying between 15% and 85%. Fig. 4 show the filtered images obtained using these filters at noise density = 95%.

Normally, there is very small difference between intensity values of adjacent pixels in a digital image. And, the central idea of our proposed filters is to copy intensity value of an uncorrupted pixel; as much adjacent as possible; to the corrupted pixel in a salt-and-pepper noise corrupted digital image. All the proposed filters first check whether a pixel is corrupted with salt-and-pepper noise; if the pixel is corrupted, the filters choose a particular pixel in the neighborhood of the corrupted pixel. If the pixel chosen in the neighborhood is itself corrupted by noise then next neighborhood pixel is chosen using a definite order defined by each filter. Each proposed filter uses a specific neighborhood of pixels and a specific order in which these pixels are chosen.

Pixels in input image (C) using Filter 1 (F1) are processed from left to right and from top to bottom (row wise). The choice of

Table 1

SSIM values of test images at varying noise densities filtered using Filter 2 (F2).

Ratio	5%	15%	25%	35%	45%	55%	65%	75%	85%	95%
Image	SSIM value									
101085	0.9716	0.9219	0.8707	0.8213	0.7633	0.7040	0.6264	0.5381	0.3951	0.1885
135069	0.9980	0.9946	0.9914	0.9874	0.9845	0.9798	0.9754	0.9667	0.9513	0.9014
145053	0.9894	0.9668	0.9405	0.9074	0.8683	0.8189	0.7522	0.6693	0.5512	0.3511
167062	0.9795	0.9585	0.9396	0.9196	0.9065	0.8849	0.8572	0.8215	0.7653	0.6596
181018	0.9894	0.9691	0.9501	0.9283	0.9059	0.8793	0.8427	0.7846	0.6992	0.5060
181079	0.9917	0.9723	0.9479	0.9171	0.8799	0.8285	0.7625	0.6680	0.5360	0.3174
189080	0.9893	0.9693	0.9507	0.9319	0.9115	0.8865	0.8504	0.7983	0.7174	0.5203
22013	0.9847	0.9556	0.9273	0.8965	0.8612	0.8251	0.7719	0.7030	0.6024	0.4175
238011	0.9963	0.9882	0.9798	0.9722	0.9641	0.9512	0.9338	0.9083	0.8573	0.7463
260058	0.9826	0.9516	0.9238	0.8961	0.8691	0.8386	0.8038	0.7567	0.6833	0.5361
23080	0.9829	0.9523	0.9222	0.8898	0.8560	0.8136	0.7639	0.6877	0.5845	0.3700
24063	0.9919	0.9800	0.9672	0.9550	0.9389	0.9258	0.9012	0.8678	0.8116	0.6621
54005	0.9753	0.9312	0.8881	0.8441	0.7990	0.7455	0.6795	0.5979	0.4761	0.2640
296059	0.9835	0.9539	0.9221	0.8875	0.8552	0.8151	0.7699	0.7063	0.6198	0.4486
302003	0.9925	0.9778	0.9628	0.9445	0.9234	0.8939	0.8516	0.7820	0.6620	0.4090
302008	0.9947	0.9833	0.9703	0.9566	0.9410	0.9202	0.8865	0.8394	0.7478	0.5384
304074	0.9781	0.9353	0.8927	0.8449	0.7940	0.7355	0.6649	0.5672	0.4321	0.2356
326038	0.9725	0.9164	0.8601	0.8024	0.7365	0.6657	0.5805	0.4747	0.3373	0.1794
35070	0.9927	0.9801	0.9689	0.9587	0.9465	0.9333	0.9128	0.8766	0.8049	0.5954
37073	0.9820	0.9493	0.9164	0.8869	0.8551	0.8204	0.7804	0.7254	0.6452	0.4862
385028	0.9659	0.9121	0.8640	0.8136	0.7647	0.7103	0.6508	0.5806	0.4823	0.3173
42049	0.9951	0.9841	0.9742	0.9617	0.9482	0.9299	0.9066	0.8741	0.8000	0.6318
97033	0.9786	0.9438	0.9079	0.8724	0.8338	0.7919	0.7333	0.6618	0.5484	0.3537
65019	0.9906	0.9728	0.9555	0.9378	0.9167	0.8934	0.8487	0.7790	0.6597	0.3822
65074	0.9894	0.9650	0.9411	0.9133	0.8848	0.8471	0.7986	0.7218	0.6079	0.3701
apple	0.9915	0.9747	0.9586	0.9416	0.9238	0.9020	0.8733	0.8326	0.7664	0.6049
camera	0.9925	0.9798	0.9681	0.9546	0.9406	0.9218	0.8942	0.8524	0.7736	0.5606
door	0.9947	0.9837	0.9720	0.9594	0.9433	0.9230	0.8962	0.8582	0.7922	0.6526
notepad	0.9927	0.9791	0.9674	0.9560	0.9439	0.9319	0.9152	0.8915	0.8528	0.7510
raindrops	0.9844	0.9576	0.9360	0.9150	0.8978	0.8713	0.8398	0.7927	0.6882	0.4563
wall	0.9794	0.9408	0.9071	0.8703	0.8358	0.7965	0.7508	0.6904	0.6034	0.4704
lena	0.9896	0.9670	0.9418	0.9151	0.8813	0.8397	0.7865	0.6808	0.5344	0.2673
barbara	0.9875	0.9607	0.9291	0.8934	0.8505	0.8012	0.7364	0.6474	0.5227	0.2828
mandril	0.9863	0.9589	0.9335	0.9045	0.8726	0.8276	0.7653	0.6656	0.5135	0.2689
cameraman	0.9934	0.9825	0.9712	0.9591	0.9442	0.9254	0.8942	0.8527	0.7663	0.6053

selecting a neighboring pixel of a corrupted pixel is done by considering the distance between a corrupted pixel and the neighboring pixel considered. Neighboring pixels as close as possible to corrupted pixel are first considered in row first order. This order of choosing neighboring pixels, however, suffers from a serious problem. In Fig. 5(a), portion of a corrupted image (C) is shown. In this image, pixels $C(i, j)$ to $C(i + 5, j)$ in column j are corrupted with either salt or pepper noise. Fig. 5(b) shows the portion of image after filtration using F1. The filter finds $C(i, j)$ corrupted with noise and copy the intensity of pixel $C(i - 1, j)$ to it using ψ_1 . Similarly, $C(i + 1, j)$ to $C(i + 5, j)$ receive intensity value of 97 from pixels right on top using ψ_1 . This undesirable propagation eventually express as vertical lines in the filtered images. These lines propagate longer with higher noise densities as can be seen in Fig. 4(b).

In order to avoid vertical noise propagation caused because of top down pixel processing using F1, in F2 pixels are processed in a bottom up manner, keeping the neighborhood and the order in which pixels in the neighborhood are considered same as in F1. Making this simple change in F2 mitigate the problem of vertical noise propagation to a large extend. Again taking example of image portion shown in Fig. 5(a); with F2, pixel at location $C(i + 5, j)$ will be processed first because of bottom up pixel processing. As the pixel $C(i + 5, j)$ is corrupted, first ψ_1 will try to choose $C(i + 4, j)$ for assignment to it but as the pixel at location $C(i + 4, j)$ is itself corrupted with noise, next ψ_2 will try to assign the corresponding pixel value to $C(i + 5, j)$. Similarly, ψ_1 will fail to choose a value for $C(i + 4, j)$, $C(i + 3, j)$, $C(i + 2, j)$ and $C(i + 1, j)$ and therefore, pixels corresponding to ψ_2 onwards may get assigned to these pixels. It may be noted however that only $C(i, j)$ shall receive the intensity value of 97 from $C(i - 1, j)$ using F2. Using F2, pixels in C are pro-

cessed from right to left and this order of pixel processing result in horizontal noise propagation in images filtered with F2. In order to understand the situation that result in horizontal noise propagation, consider the portion of a noise corrupted image in Fig. 5(c). In this figure, pixel $C(i, j + 1)$ has a value 100 and let $C(i, j)$ is the next pixel that needs to be processed for noise filtering. As $C(i, j)$ is corrupted with noise, first ψ_1 try to choose $C(i - 1, j)$ for assignment to $C(i, j)$ but as the pixel $C(i - 1, j)$ is itself corrupted, ψ_2 try to choose $C(i, j - 1)$ but again this pixel is also corrupted with noise. Finally, ψ_3 successfully choose $C(i, j + 1)$ with intensity value 100 for assignment to $C(i, j)$. Similarly, for cells $C(i, j - 1)$ to $C(i, j - 5)$, in row i moving from right to left, both ψ_1 and ψ_2 fail (assuming $C(i, j - 6)$ is also corrupted) because for these pixels, pixels located on top as well as left are noise corrupted. So, using ψ_3 intensity value of 100 is propagated to all these cells as shown in Fig. 5(d). This effect is clearly visible in Fig. 4(c); horizontal lines running from right to left are visible but there are no vertical lines in the figure. Although both vertical and horizontal noise propagation affect the quality of filtered images but while for vertical noise propagation n pixels need to be corrupted in a column to span a vertical noise of n pixels, it takes $2n$ number of pixels in two adjacent rows to create a horizontal noise spanning n pixels. Therefore, horizontal noise propagation is less likely to happen than vertical noise propagation.

As both top to bottom as well as right to left pixel processing cause vertical and horizontal noise propagation respectively, F3 try to remedy both these problems by using bottom up pixel processing to avoid vertical noise propagation and selecting only two rules ψ_1 and ψ_2 so that no horizontal noise propagation is possible. This approach is simple and can provide good performance when



Fig. 3. Noise corrupted and filtered images with noise density ranging between 15%–85%.

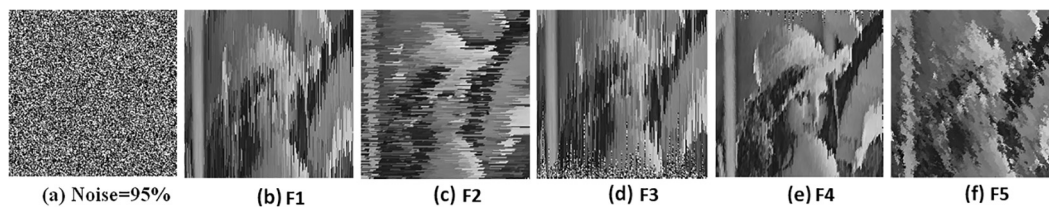


Fig. 4. Noise corrupted and filtered images at 95% noise density.

the noise density is relatively low in the input image. But as the noise density increase, two major problems reduce the filtering capabilities of this filter to a great extend. First, choice of considering only two neighboring pixels leaves many corrupted pixels

unchanged near bottom of the image as can be seen in Fig. 4(d). This happens because, if for a corrupted pixel, both top and bottom pixels are corrupted; the corrupted pixel is left unchanged. Second problem that arises at high noise density is problem of vertical

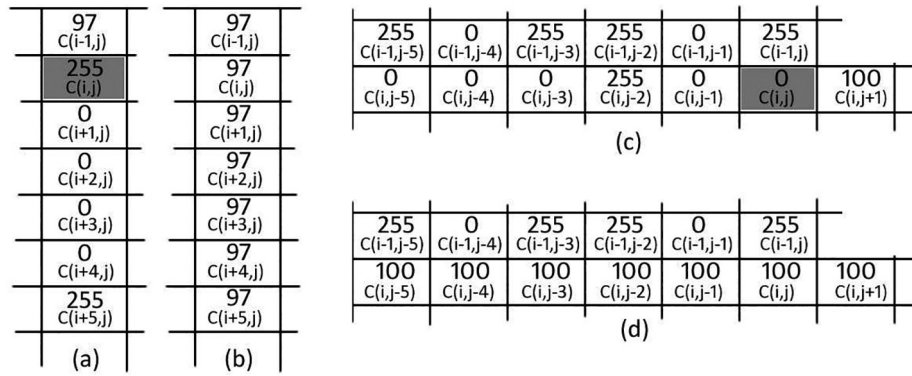


Fig. 5. Horizontal & Vertical noise propagation (a) Pixel values before filtration (b) Pixel values after filtration (c) Pixel values before filtration (d) Pixel values after filtration.

noise propagation; considering a situation wherein n number of pixels are corrupted in a column and $(n + 1)$ th pixel is uncorrupted, primary rule ψ_1 will fail for all these n pixels and ψ_2 will copy value of $(n + 1)$ th pixel to these n pixels above. It should be understood here that in F1 noise propagate from top to bottom while as in F3, it propagates in the opposite direction.

In F4, approach opposite to F3 is taken with regard to neighboring pixels. In F4, 24 neighboring pixels represented by primary rules ψ_1 to ψ_{24} are used so that even at high noise densities, a pixel is assigned value of an uncorrupted pixel in local or non-local neighborhood. Use of 16 non-local pixels to a corrupted pixel $C(i, j)$, however, results in slight distortion of the image. Use of ψ_6 is forced if last pixel was processed using ψ_2 to avoid horizontal noise propagation from left to right. Therefore, horizontal noise propagation is controlled to a great extent but vertical noise propagation still happen, although less severe, because ψ_1 cause vertical noise propagation in the same manner it does in F1. In F5, primary rules ψ_1 to ψ_{12} are used such that if pixel $C(i, j)$ is treated using ψ_k , then the pixel at location $C(i, j + 1)$ is treated using ψ_l where $l = k + 1$. Reason behind using such an ordering is to avoid both horizontal as well as vertical noise propagation by making sure that no primary rule assigns a value continuously in a column or row. But, again this ordering fails to provide desired results at very high noise density. The output of filtering of *lena* image with 95% noise is shown in Fig. 4(f). It is interesting that F5 give a distorted appearance to this image as if filtered using an artistic filter.

4.1. SSIM analysis of proposed methods

Noise filtering performance of proposed filters is first analyzed using SSIM metric. SSIM values for all the test images at varying noise densities is recorded after filtering using all the proposed filters. Table 1 list the SSIM values achieved for the test images after noise filtration using F2. It is evident from the table that SSIM values get better with decrease in noise ratio. SSIM values achieved for all test images, at small noise ratios, is nearly same but SSIM values show great variation as noise densities increase in test

images. This variation may be attributed to the difference in texture and size of test images.

For the want of space, it isn't possible to represent the SSIM values obtained for every single image by using rest of the proposed filters here. However, in order to present comparative performance of the proposed filters, we computed average of SSIM values for all test images at varying noise density ratios as presented in Table 2. Fig. 7(a) shows the values presented in Table 2 as a line graph. From Fig. 7(a), it can be observed that Filter 2 (F2) produce better SSIM values compared to other filters at all noise densities. SSIM values decrease almost linearly for all the filters with increase in noise density. The decrease is, however, more steep as the noise density increase from 85% to 95%.

4.2. PSNR analysis of proposed methods

Same set of 35 test images was used to perform PSNR analysis of the proposed filters. PSNR value for each image after filtration of images corrupted with different noise ratios is recorded for each filter. Average PSNR values achieved with these filters are presented in Table 3. Fig. 7(b) provide the graphical representation of the data presented in Table 3. It is evident from this figure that F2 perform better than all other filters at all noise ratios and the PSNR achieved with F2 is clearly much better at high noise ratios. F5 result in smallest PSNR values till 55% noise and at higher noise ratios F5 shows a transition and is shown to perform better than F1 and F3. Almost same results are suggested by average SSIM values for the set of test images.

4.3. Comparative analysis of proposed methods

In order to analyze comparative performance of the proposed filters, three standard salt-and-pepper noise filters namely Arithmetic Filter (AF), Median Filter (MF) and Switching Median Filter (SMF) and two CA-based filters proposed by Selvapeter and Hordijk (2009) and Qadir and Shoosha (2018) are used. Fig. 6 provides a visual comparison of filtering capabilities of standard, CA-based and proposed (F2 and F4) filters. Same set of test images

Table 2
Average SSIM values of test images at varying noise densities obtained using proposed filters.

Ratio	5%	15%	25%	35%	45%	55%	65%	75%	85%	95%
Filter	SSIM value									
F1	0.9847	0.9553	0.9212	0.8819	0.8369	0.7835	0.7199	0.6408	0.5359	0.3506
F2	0.9866	0.9620	0.9377	0.9119	0.8841	0.8508	0.8074	0.7463	0.6512	0.4659
F3	0.9863	0.9601	0.9309	0.8968	0.8566	0.8065	0.7434	0.6609	0.5425	0.3195
F4	0.9857	0.9586	0.9288	0.8972	0.8622	0.8214	0.7720	0.7066	0.6141	0.4299
F5	0.9798	0.9417	0.9027	0.8624	0.8190	0.7732	0.7198	0.6550	0.5710	0.4343

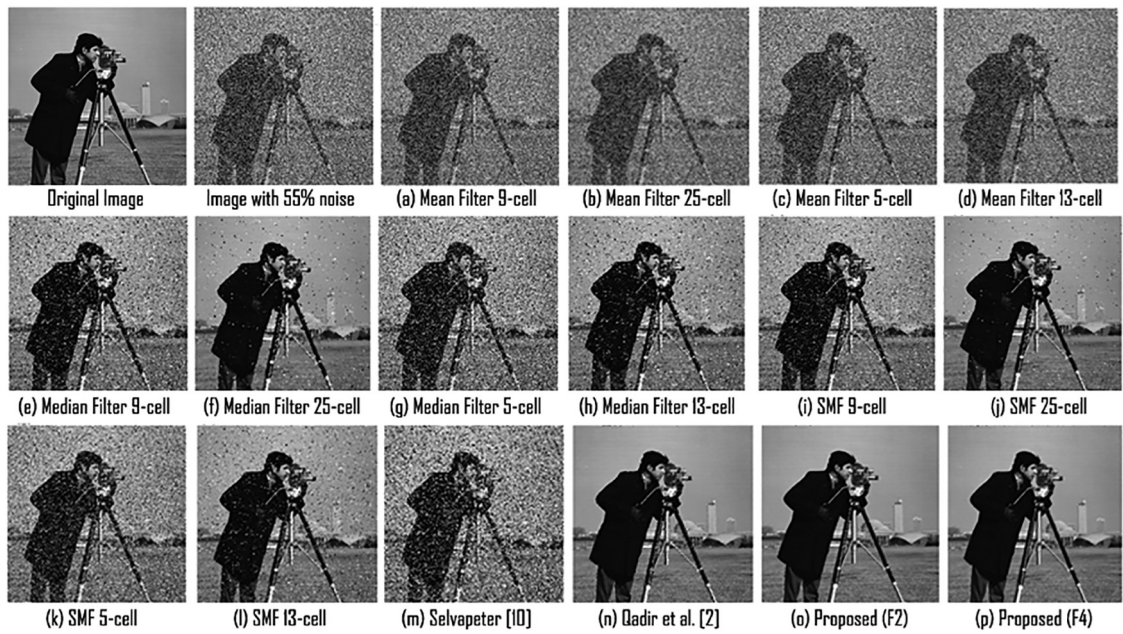


Fig. 6. Visual comparison of proposed filters with different standard and CA-based filters using cameraman test image corrupted with 55% noise.

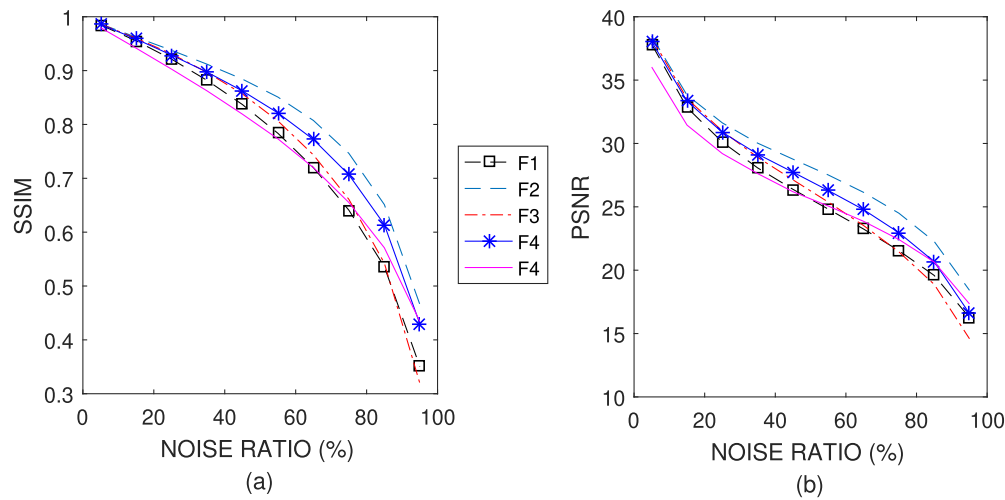


Fig. 7. Comparative performance of proposed filters at varying noise levels. (a) Plot of average SSIM values. (b) Plot of average PSNR values.

with different noise ratios is filtered using the filters. Average SSIM and average PSNR at varying noise ratios for all these filters and the proposed filter (F2) is given in Table 4 and Table 5 respectively. From the comparison of values in Table 2 with 4 and Table 3 with 5, it can be observed that all proposed filters provide better filtration at higher noise density.

In Table 4, AF and SMF provide better results using N_M with $r = 2$ (25 pixels) followed by N_{vN} with $r = 2$ (13 cells) followed by N_M with $r = 1$ (9 pixels) and then N_{vN} with $r = 1$ (5 cells). Same results are shown by MF beyond 35% noise density. An observation about these filters as per our experimental results reveal that these filters perform better when the number of pixels involved is

Table 3
Average PSNR values of test images at varying noise densities obtained using proposed filters.

Ratio	5%	15%	25%	35%	45%	55%	65%	75%	85%	95%
Filter	PSNR value									
F1	37.8308	32.8314	30.1098	28.0728	26.3820	24.8200	23.2482	21.5449	19.5886	16.2635
F2	38.4608	33.8150	31.6242	30.0264	28.7625	27.5159	26.1432	24.4988	22.2710	18.4108
F3	38.3216	33.5338	30.9856	28.9025	27.1318	25.3768	23.5251	21.4486	18.8825	14.5897
F4	38.0323	33.3345	30.8845	29.1580	27.7359	26.2900	24.7776	22.9627	20.6746	16.5842
F5	36.0289	31.4487	29.2021	27.6244	26.1913	25.0633	23.8581	22.4226	20.6446	17.3379

Table 4

Average SSIM values of test images at varying noise densities obtained using proposed (F2) and other standard filters.

Ratio	5%	15%	25%	35%	45%	55%	65%	75%	85%	95%
Filter	SSIM value									
AF ($N_M, r = 1$)	0.5728	0.3547	0.2586	0.2001	0.1580	0.1255	0.0988	0.0770	0.0583	0.0421
AF ($N_M, r = 2$)	0.6207	0.4701	0.3792	0.3159	0.2668	0.2271	0.1928	0.1635	0.1387	0.1158
AF ($N_{vN}, r = 1$)	0.5300	0.2876	0.1981	0.1474	0.1127	0.0865	0.0657	0.0490	0.0350	0.0231
AF ($N_{vN}, r = 2$)	0.5951	0.3939	0.2963	0.2341	0.1887	0.1530	0.1235	0.0991	0.0780	0.0593
MF ($N_M, r = 1$)	0.8830	0.8574	0.7688	0.5727	0.3337	0.1650	0.0780	0.0387	0.0189	0.0079
MF ($N_M, r = 2$)	0.7832	0.7757	0.7649	0.7489	0.7057	0.5742	0.3340	0.1252	0.0379	0.0117
MF ($N_{vN}, r = 1$)	0.9266	0.7731	0.4916	0.2613	0.1361	0.0760	0.0440	0.0257	0.0143	0.0066
MF ($N_{vN}, r = 2$)	0.8518	0.8405	0.8108	0.7146	0.5142	0.2791	0.1222	0.0520	0.0226	0.0085
SMF ($N_M, r = 1$)	0.9539	0.9399	0.8701	0.6949	0.4455	0.2363	0.1137	0.0549	0.0256	0.0095
SMF ($N_M, r = 2$)	0.9667	0.9631	0.9433	0.9130	0.8564	0.7133	0.4416	0.1769	0.0527	0.0139
SMF ($N_{vN}, r = 1$)	0.9529	0.8186	0.5501	0.3093	0.1656	0.0930	0.0539	0.0314	0.0173	0.0076
SMF ($N_{vN}, r = 2$)	0.9574	0.9549	0.9309	0.8471	0.6556	0.3917	0.1823	0.0764	0.0313	0.0104
Selvapeter and Hordijk (2009)	0.9590	0.7613	0.4938	0.2725	0.1395	0.0733	0.0399	0.0225	0.0129	0.0067
Qadir and Shoosha (2018)	0.9860	0.9589	0.9279	0.8921	0.8505	0.8004	0.7391	0.6595	0.5513	0.3562
Proposed (F2)	0.9866	0.9620	0.9377	0.9119	0.8841	0.8508	0.8074	0.7463	0.6512	0.4659

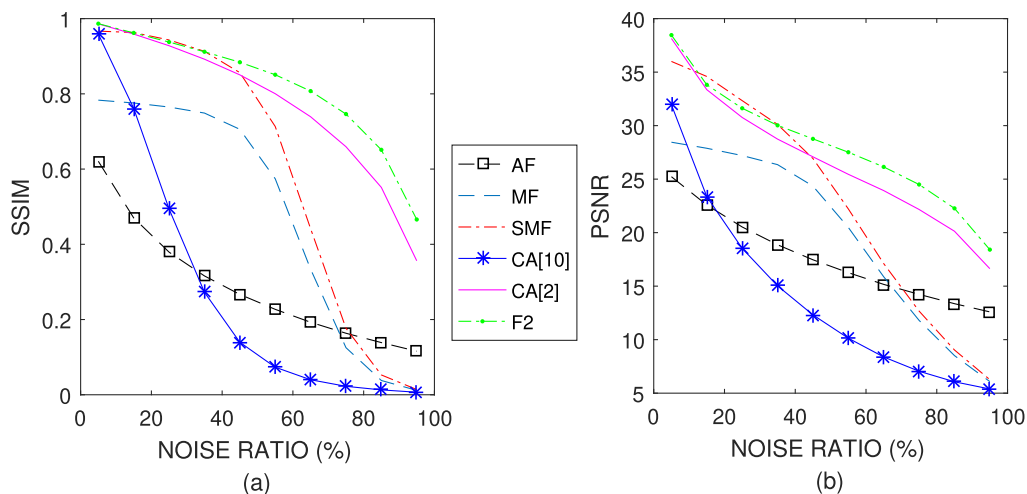
Table 5

Average PSNR values of test images at varying noise densities obtained using proposed (F2) and other standard filters.

Ratio	5%	15%	25%	35%	45%	55%	65%	75%	85%	95%
Filter	PSNR value									
AF ($N_M, r = 1$)	25.0441	21.1686	18.8686	17.1845	15.8316	14.7083	13.7063	12.8568	12.0627	11.3677
AF ($N_M, r = 2$)	25.2271	22.5798	20.5289	18.8677	17.4632	16.2605	15.1646	14.2341	13.3622	12.5945
AF ($N_{vN}, r = 1$)	24.0585	19.5739	17.2217	15.5814	14.2961	13.2424	12.3205	11.5339	10.8104	10.1741
AF ($N_{vN}, r = 2$)	25.4024	21.8974	19.6428	17.9371	16.5435	15.3732	14.3277	13.4380	12.6060	11.8776
MF ($N_M, r = 1$)	31.7269	29.3088	25.0649	20.4601	16.5177	13.3392	10.7511	8.6964	7.0258	5.7005
MF ($N_M, r = 2$)	28.4502	27.8844	27.2035	26.3551	24.3485	20.4863	15.8756	11.8012	8.5106	6.0816
MF ($N_{vN}, r = 1$)	32.8961	24.9456	19.5224	15.7818	12.9969	10.8454	9.0981	7.6974	6.5271	5.5611
MF ($N_{vN}, r = 2$)	30.6862	29.6716	27.6306	23.8487	19.5047	15.5249	12.2079	9.5686	7.4502	5.8127
SMF ($N_M, r = 1$)	35.8313	32.5878	27.2750	22.1401	17.9040	14.5107	11.7062	9.4099	7.4684	5.8530
SMF ($N_M, r = 2$)	35.9994	34.5932	32.3360	30.1089	26.9260	22.2162	17.1071	12.6705	9.0368	6.2605
SMF ($N_{vN}, r = 1$)	34.7768	25.9531	20.3145	16.5040	13.6620	11.4427	9.6135	8.1077	6.8006	5.6626
SMF ($N_{vN}, r = 2$)	35.9444	34.4879	31.1493	26.2130	21.2583	16.9115	13.2989	10.3596	7.9349	5.9777
Selvapeter and Hordijk (2009)	31.9506	23.3605	18.5305	15.0289	12.3166	10.1368	8.3813	7.0489	6.0641	5.3900
Qadir and Shoosha (2018)	38.1452	33.3594	30.7633	28.7403	27.0852	25.4514	23.9301	22.1665	20.1334	16.6333
Proposed (F2)	38.4608	33.8150	31.6242	30.0264	28.7625	27.5159	26.1432	24.4988	22.2710	18.4108

greater. In Fig. 8(a), we present plot of 25-pixel versions of AF, MF, SMF versions only, two CA-based filters and proposed F2 filter in order to keep the plot less cluttered. It can be observed from the figure that proposed filter (F2) provides better results in terms of SSIM at all noise densities. SMF is shown to perform

slightly better at low noise densities compared to F2 but beyond 45% there is a sharp decline in SSIM values obtained using SMF while F2 maintain to result in comparatively better SSIM. Fig. 8 (b) is the plot of average PSNR values obtained using filters presented in Table 5. As with SSIM, the PSNR obtained using SMF is

**Fig. 8.** Comparative performance of different filters at varying noise levels. (a) Plot of average SSIM values. (b) Plot of average PSNR values.

slightly better than those of F2 but beyond 35% noise density SMF perform poor in terms of PSNR when compared to F2.

5. Conclusion

In this paper, a computationally efficient noise filtration method based on OTCA, under five different neighborhood conditions is presented, capable of noise reduction in digital images effectively across a wide range of noise densities varying up to 85%. Among the proposed filters, Filter 2 (F2) results in best noise filtration both subjectively as well as objectively in terms of SSIM and PSNR. The proposed methods use comparison and assignment operations only for noise filtration. Computational efficiency, simplicity, robustness and exploitation of the parallel processing in CA for noise filtration are the main advantages of the proposed method. Comparison with the conventional methods reveals a significant improvement in noise filtration capability using proposed filters. Using other subsets of von Neumann and Moore neighborhood at higher values of *r* in different combinations and orders for eliminating specific problems in high noisy images are the concern of authors for more future research.

Declarations of interest

None.

Funding source(s)

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Acronym	Description
OTCA	Outer totalistic cellular automata
TCA	Totalistic cellular automata
FCA	Fuzzy cellular automata
AF	Arithmetic filter
MF	Median filter
WMF	Weighted median filter
CWMF	Center weighted median filter
SMF	Switching median filter
N _M	Moore neighborhood
N _{VN}	Von Neumann Neighborhood
2D-CA	Two dimentional cellular automata

References

Thepade, S., Das, R., Ghosh, S., 2017. Decision fusion-based approach for content-based image classification. *Int. J. Intelligent Comput. Cybern.* 10 (3), 310–331. <https://doi.org/10.1108/ijicc-07-2016-0025>.

Qadir, F., ShooSha, I.Q., 2018. Cellular automata-based efficient method for the removal of high-density impulsive noise from digital images. *Int. J. Inf. Technol.* 10 (4), 529–536. <https://doi.org/10.1007/s41870-018-0166-4>.

Singh, P., Shree, R., Diwakar, M., 2018. A new SAR image despeckling using correlation based fusion and method noise thresholding. *J. King Saud Univ.-Comput. Inf. Sci.* <https://doi.org/10.1016/j.jksuci.2018.03.009>.

Sahin, U., Uguz, S., Sahin, F., 2014. Salt and pepper noise filtering with fuzzy-cellular automata. *Comput. Electr. Eng.* 40 (1), 59–69. <https://doi.org/10.1016/j.compeleceng.2013.11.010>.

Tukey, J., 1974. Nonlinear (nonsuperposable) methods for smoothing data. In: *Electronic and Aerospace Systems Conference*, pp. 673–681.

Brownrigg, D.R.K., 1984. The weighted median filter. *Commun. ACM* 27 (8), 807–818. <https://doi.org/10.1145/358198.358222>.

Ko, S.-J., Lee, Y., 1991. Center weighted median filters and their applications to image enhancement. *IEEE Trans. Circuits Syst.* 38 (9), 984–993. <https://doi.org/10.1109/31.83870>.

Sun, T., Neuvo, Y., 1994. Detail-preserving median based filters in image processing. *Pattern Recogn. Lett.* 15 (4), 341–347. [https://doi.org/10.1016/0167-8655\(94\)90082-5](https://doi.org/10.1016/0167-8655(94)90082-5).

Liu, S., Chen, H., Yang, S., 2008. An effective filtering algorithm for image salt-pepper noises based on cellular automata. In: *2008 Congress on Image and Signal Processing*. IEEE. <https://doi.org/10.1109/cisp.2008.263>.

Selvapeter, P.J., Hordijk, W., 2009. Cellular automata for image noise filtering. In: *2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*. IEEE. <https://doi.org/10.1109/nabic.2009.5393684>.

Dalhoun, A., Al-Dhamari, I., Ortega, A., Alfonsoeca, M., 2011. Enhanced cellular automata for image noise removal. In: *Proceedings of the Asian Simulation Technology Conference*, pp. 69–73.

Sadeghi, S., Rezvanian, A., Kamrani, E., 2012. An efficient method for impulse noise reduction from images using fuzzy cellular automata. *AEU – Int. J. Electr. Commun.* 66 (9), 772–779. <https://doi.org/10.1016/j.aeue.2012.01.010>.

Shukla, A.P., Agarwal, S., 2014. An enhanced cellular automata based scheme for noise filtering. *Int. J. Signal Process. Image Process. Pattern Recogn.* 7 (4), 231–242. <https://doi.org/10.14257/ijisp.2014.7.4.23>.

Sargolzaei, A., Yen, K.K., Zeng, K., Motahari, S.M.A., Noei, S., 2014. Impulse image noise reduction using fuzzy-cellular automata method. *Int. J. Comput. Electr. Eng.* 6 (2), 191–195. <https://doi.org/10.7763/ijcee.2014.v6.820>.

Pathak, M., Sadawarti, H., Singh, S., 2016. A technique to suppress speckle in ultrasound images using nonlocal mean and cellular automata. *Indian J. Sci. Technol.* 9 (13). <https://doi.org/10.17485/ijst/2016/v9i13/80421>.

Tourtounis, D., Mitianoudis, N., Sirakoulis, G.C. Salt-n-pepper noise filtering using cellular automata. *CoRR abs/1708.05019*. arXiv:1708.05019.

Roy, A., Singha, J., Devi, S.S., Laskar, R.H., 2016. Impulse noise removal using SVM classification based fuzzy filter from gray scale images. *Signal Process.* 128, 262–273. <https://doi.org/10.1016/j.sigpro.2016.04.007>.

Kumar, S.V., Nagaraju, C. Support vector neural network based fuzzy hybrid filter for impulse noise identification and removal from gray-scale image, *J. King Saud Univ.- Comput. Inf. Sci.*<https://doi.org/10.1016/j.jksuci.2018.05.011>.

Preston Jr., K., Duff, M.J., 1984. *Modern Cellular Automata: Theory and Applications*. Plenum, New York.

Halbach, M., Hoffmann, R., 2004. Implementing cellular automata in fpga logic. In: *Parallel and Distributed Processing Symposium, 2004. Proceedings. 18th International*. IEEE, pp. 258–262. <https://doi.org/10.1109/IPDPS.2004.1303324>.

Toffoli, T., Margolus, N., 1987. *Cellular Automata Machines: A New Environment for Modeling*. MIT Press.

Jeelani, Z., Qadir, F., 2018. Cellular automata-based approach for digital image scrambling. *Int. J. Intelligent comput. cybern.* 11 (3), 353–370. <https://doi.org/10.1108/ijicc-10-2017-0132>.

Packard, N.H., Wolfram, S., 1985. Two-dimensional cellular automata. *J. Stat. Phys.* 38 (5), 901–946. <https://doi.org/10.1007/bf01010423>.

Dalhoun, A.L.A., Mahafzah, B.A., Awwad, A.A., Aldhamari, I., Ortega, A., Alfonsoeca, M., 2012. Digital image scrambling using 2d cellular automata. *IEEE MultiMedia* 19 (4), 28–36. <https://doi.org/10.1109/mmul.2011.54>.

Manzoni, L., 2012. Asynchronous cellular automata and dynamical properties. *Nat. Comput.* 11 (2), 269–276. <https://doi.org/10.1007/s11047-012-9308-y>.